

简历深度面试解答手册 — 汪忠孝

本文档针对简历中每一条技能点、项目细节进行逐条深挖，以「面试官可能怎么问 + 你应该怎么答」的形式呈现。

一、专业技能逐条深挖

1.1 语言基础：JavaScript (ES6+) / TypeScript / HTML5 / CSS3 / Sass / Less

Q：ES6+ 你常用哪些特性？举几个实际场景。

A：

- **解构赋值**：从接口返回的对象中提取字段，如 `const { name, price, images } = product`，避免重复写 `product.xxx`
- ****可选链 ?.** 与空值合并 `??`：处理接口数据缺失，如 `supplier?.company?.name ?? '未知供应商'`
- **Promise.all / Promise.allSettled**：在首页多楼层并行请求时，用 `Promise.allSettled` 保证单个楼层接口失败不影响其他楼层渲染
- **Proxy / Reflect**：在 Vue 3 响应式原理中理解其底层机制；在表单场景中用 Proxy 做数据变更拦截
- **WeakMap / WeakSet**：在做组件缓存时用来存储不阻止垃圾回收的引用，比如缓存 DOM 节点映射
- **Iterator / for...of / Generator**：在轨迹回放功能中，用 Generator 逐帧产出轨迹点数据，控制播放节奏

Q：TypeScript 在项目里怎么用的？泛型用过吗？

A：

- 在 Alibaba.com 全球供给项目中，React + TS 是主技术栈。所有楼层组件的 props、接口返回数据、GraphQL Query 结果都有严格类型定义
- 泛型简单说就是“类型参数”——写代码时不写死具体类型，用的时候再指定，让代码既灵活又安全。
- 泛型的典型场景：封装通用 API 请求函数 `function fetchData<T>(query: string): Promise<ApiResponse<T>>`，T 根据楼层数据类型动态指定
- 用 `Record<string, FloorConfig>` 做楼层配置映射，用 `Partial<T>` 做编辑表单的类型推导
- 在 GraphQL 场景下，通过 codegen 工具根据 Schema 自动生成 TS 类型，保证前后端类型一致

Q：Sass 和 Less 的区别是什么？项目里怎么选？

A：

- Sass 用 `@mixin / @include` 做样式复用，支持函数、条件判断，生态更成熟（Element Plus 用的就是 Sass）
- Less 语法更接近 CSS，用 `@mixin` 较少，主要靠 `mixin` 函数，Ant Design 使用 Less 做主题定制
- 在 OMS 项目中用 Sass 嵌套语法和变量管理样式；在大华时期 View UI 组件库基于 Less，所以也用 Less 做主题变量覆盖

- 两者核心差异不大，关键是跟着组件库的生态走

1.2 前端框架：精通 Vue 2/3 全家桶（Vue Router / Pinia / Vuex），熟练 React 18 + Hooks 生态（ice.js）

Q：Vue 2 和 Vue 3 的核心区别？

A：

- 响应式原理：Vue 2 用 `Object.defineProperty` 劫持 `getter/setter`，Vue 3 用 `Proxy` 代理整个对象，支持 `Map/Set/WeakMap` 等数据结构，且可以监听属性新增和删除
- **Composition API**：Vue 3 提供 `setup()` + `ref/reactive/computed/watch`，可以把相关逻辑聚合在一起（而不是按 `data/methods/computed` 分散），利于逻辑复用和代码组织
- 生命周期变化：`beforeCreate/created` 被 `setup()` 替代，`beforeDestroy/destroyed` 改为 `beforeUnmount/unmounted`
- 性能优化：Vue 3 引入静态提升（`hoistStatic`）、补丁标记（`PatchFlags`）、Block Tree，diff 算法只比较动态节点，渲染性能提升显著
- **Tree-shaking**：Vue 3 全局 API 改为按需导入（如 `import { createApp } from 'vue'`），打包体积更小

Q：Pinia 和 Vuex 的区别？为什么选 Pinia？

A：

- Pinia 是 Vue 官方推荐的状态管理，去掉了 Vuex 的 `mutations`，只有 `state/getters/actions`，更简洁
- 天然支持 TypeScript，state 类型自动推导
- 模块化不需要 `modules` 嵌套，每个 store 独立文件，按需引入
- 支持插件扩展（如持久化插件 `pinia-plugin-persistedstate`）
- 在 OMS 项目中从 Vuex 迁移到 Pinia，store 文件数量没变，但代码量减少约 30%，去掉了大量 `commit` 的 `mutations` 样板代码

Q：Vue Router 路由守卫你怎么用的？

A：

- ****全局前置守卫 `router.beforeEach`****：做登录鉴权，检查 token 是否存在，不存在跳转登录页；检查角色权限，无权限跳转 403
- ****路由独享守卫 `beforeEnter`****：特定路由需要额外校验，如设备详情页需要先验证设备 ID 合法性
- ****组件内守卫 `onBeforeRouteLeave`****（Vue 3）：表单编辑页离开前提示用户"是否保存未保存的更改"
- 在 OMS 项目中，结合 OA 单点登录的 token，在全局守卫中做 token 有效性校验 + 自动刷新

Q：React 18 的 Hooks 你用哪些？和 Vue 3 Composition API 有什么区别？

A：

- 常用：`useState`、`useEffect`、`useMemo`、`useCallback`、`useRef`、`useContext`、自定义 Hooks

- 在 Alibaba.com 项目中, ice.js 框架提供 `useData`、`useSearchParams` 等内置 Hooks, 结合 React 18 的 `useTransition` 做非阻塞渲染 (比如切换楼层 Tab 时标记为低优先级更新)
 - 核心区别:
 - Vue 3 的 `ref/reactive` 是真正的响应式 (赋值即触发更新), React 的 `useState` 需要调用 `setState` 手动触发
 - Vue 3 的 `computed` 自动追踪依赖, React 的 `useMemo` 需要手动声明依赖数组
 - Vue 3 的 `watch` 可以精确监听某个属性变化, React 的 `useEffect` 是依赖整体对比
 - React Hooks 有闭包陷阱 (stale closure), 需要用 `useRef` 或 `useCallback` 解决; Vue 3 因为是响应式代理, 没有这个问题
-

1.3 UI 组件库: Element / Element Plus / Ant Design / Fusion Design / View UI

Q: 组件库二次封装你是怎么做的? 举个例子。

A:

- 在 OMS 项目中封装了 G 系列全局业务组件, 比如 `GTable` (增强表格):
 - 基于 Element Plus 的 `el-table`, 内置了分页器、列配置面板、导出 Excel 按钮、表格列拖拽排序
 - 通过 `props.columns` 配置驱动渲染, 传入列定义数组即可生成完整表格
 - 统一了 loading 状态管理 (内置 `v-loading`)、空数据占位图、错误兜底
- 封装原则: 透传不覆盖。原组件的所有 props/slots/events 通过 `v-bind="$attrs"` 透传, 新功能通过额外 props 叠加
- 用 `defineExpose` 暴露内部方法 (如 `refresh()`、`getSelectedRows()`), 方便父组件调用

Q: Fusion Design 和 Element Plus 有什么区别?

A:

- Fusion Design 是阿里 UED 团队出品, 主要服务阿里系产品, 设计风格偏 B 端国际化, 在 ice.js 框架中有原生支持
 - Element Plus 是饿了么团队出品, 社区生态更大, 中文文档完善, 适合国内 B 端管理系统
 - Fusion Design 的组件 API 设计上更偏向可配置化 (如 `ConfigProvider` 全局配置), 且对国际化 (i18n) 支持更好
 - 在 Alibaba.com 项目中用 Fusion Design 是因为项目面向全球买家, 需要多语言支持和 RTL (从右到左) 布局支持
-

1.4 工程化: Vite / Webpack / pnpm / ESLint / 按需导入 / 路由懒加载 / 分包策略

Q: Vite 比 Webpack 快在哪里?

A:

- 开发环境: Vite 基于原生 ESM (ESM 是 JavaScript 官方的标准模块系统, 也就是你用 `import / export` 写代码时用的那套规范。), 不做打包, 浏览器直接请求模块文件, 启动速度几乎是毫秒级 (Webpack 需要先打包所有模块)
- 热更新 HMR: Vite 只更新变化的模块 (利用 ESM 的 `import.meta.hot`), Webpack 需要重新编译整个模块依赖图

- **预构建**：Vite 用 esbuild 做依赖预构建（将 CommonJS/UMD 转为 ESM），速度比 Webpack 快 10-100 倍
- **生产构建**：Vite 用 Rollup 打包，输出 Tree-shaking 效果更好的 ESM 格式

Q：按需导入你是怎么做的？打包体积减少了多少？

A：

- Element Plus 按需导入：用 `unplugin-vue-components` + `unplugin-auto-import` 插件，自动扫描模板中使用的组件并按需导入对应样式
- 原理：插件在编译时解析 `<el-button>` 等标签，自动注入 `import { ElButton } from 'element-plus'` 和对应 CSS
- 不需要手动写 `import` 语句，也不需要再 `main.js` 中全量引入
- 在 OMS 项目中，从全量引入 Element Plus（约 700KB CSS）改为按需导入后，CSS 体积减少约 80%

Q：路由懒加载怎么实现的？

A：

```
// 不使用懒加载（同步导入，所有页面打包在一个 chunk 里）
import DeviceManage from "@views/device/index.vue";

// 使用懒加载（动态 import，访问时才加载）
const DeviceManage = () =>
  import(/* webpackChunkName: "device" */ "@views/device/index.vue");
```

- 在 Vite 中直接使用 `() => import('./views/xxx.vue')`，Vite 会自动做代码分割
- 原理：`import()` 是 ES 的动态导入语法，返回 Promise，Webpack/Vite 会将该模块单独打成一个 chunk
- 结合 `webpackChunkName` 魔法注释（Webpack）或 Rollup 的 `manualChunks`（Vite），可以控制分包粒度

Q：分包策略具体怎么配的？说说你的思路。

A：在 Vite 中通过 `build.rollupOptions.output.manualChunks` 配置：

```
// vite.config.js
export default {
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          "vendor-vue": ["vue", "vue-router", "pinia"], // Vue 核心库
          "vendor-ui": ["element-plus"], // UI 组件库
          "vendor-utils": ["axios", "dayjs", "lodash-es"], // 工具库
          "vendor-map": ["ol", "proj4"], // 地图库（体积大，单独分包）
        },
      },
    },
  },
}
```

```
    },  
  },  
};
```

分包思路：

1. **核心框架单独分包** (vue/vue-router/pinia)：这些库几乎不变，利用浏览器缓存，用户首次加载后后续访问直接用缓存
2. **UI 组件库单独分包** (element-plus)：体积大且更新频率低于业务代码
3. **重型库单独分包** (OpenLayers、ECharts)：只在特定页面使用，避免加载到首屏
4. **业务代码按路由自动分割**：结合路由懒加载，每个页面自动一个 chunk
5. **效果**：首屏只需加载 `vendor-vue + vendor-ui + 当前页面 chunk`，从原本的 23M 单文件降至首屏仅加载约 2M 的核心包

Q：pnpm 和 npm/yarn 的区别？为什么选 pnpm？

A：

- **硬链接 + 符号链接**：pnpm 将依赖安装到全局 store (`~/pnpm-store`)，项目 `node_modules` 里只有符号链接，节省磁盘空间
- **严格的依赖隔离**：pnpm 不允许访问未在 `package.json` 中声明的依赖（幽灵依赖），避免"我这里没装但能用"的隐患
- **安装速度**：得益于硬链接机制，安装速度比 npm/yarn 快 2-3 倍
- **Monorepo 支持**：通过 `pnpm workspace` 管理多包项目，比 Lerna 更轻量

1.5 可视化：OpenLayers / Leaflet / 高德地图 / WebGL / Canvas / ECharts / Recharts

Q：OpenLayers 和 Leaflet 的区别？为什么不同项目用不同的？

A：

- **OpenLayers**：功能全面，支持多坐标系（通过 proj4 转换）、多图层叠加、高级渲染（WebGL 图层）、复杂交互（绘制、测量），适合专业 GIS 场景
- **Leaflet**：轻量（约 40KB），API 简洁，适合简单的地图展示和标记，插件生态丰富
- **OMS 项目用 OpenLayers 因为需要**：多坐标系转换（WGS84 → GCJ02）、10W+ 海量点位渲染、轨迹回放动画、多图层管理
- **全球机会洞察平台用 Leaflet 因为只需要全球营销活动分布的简单地图视图**，轻量即可

Q：WebGLPointsLayer 怎么做到 10W+ 点位渲染的？FPS 如何保证？

A：

- OpenLayers 默认的 Canvas 渲染器在点位超过 1W 时就会明显卡顿，因为每个 Feature 都是独立 DOM 或 Canvas 绘制调用
- `WebGLPointsLayer` 使用 WebGL 直接在 GPU 上批量渲染所有点位，一次 `drawCall` 绘制所有点
- **关键优化手段**：
 1. **视口裁剪**：只渲染当前视口内的点位，地图移动时动态更新可视区域
 2. **LOD (Level of Detail)**：缩放级别低时聚合显示（Cluster），缩放级别高时展开单个点

3. **属性缓冲区**：点位数据（坐标、颜色、大小）预计算存入 WebGL Buffer，避免每帧重复计算

4. **Web Worker 预处理**：坐标转换（proj4）放在 Worker 线程中，不阻塞主线程

- **FPS 保障**：10W+ 点位在 WebGL 模式下 FPS 保持 10+（在中等配置电脑上），Canvas 模式下同样的数据量 FPS 只有 1-2

Q：轨迹回放怎么做的？速度/方向动画怎么实现？

A：

核心思路：按时间轴插值轨迹点 → 更新标记位置 → 计算朝向角 → 旋转图标

- **数据准备**：后端返回带时间戳的轨迹点数组 `[{lon, lat, speed, timestamp, heading}]`
- **插值算法**：根据播放倍速计算当前时间点，用线性插值在两个相邻轨迹点之间计算当前位置
- **方向动画**：通过 `heading`（航向角）或相邻两点计算方位角 `Math.atan2(dy, dx)`，设置图标旋转角度
- **速度展示**：用颜色渐变表示速度变化（绿色→黄色→红色），或在线段上叠加速度-时间折线图
- **性能**：使用 `requestAnimationFrame` 驱动动画循环，避免 `setInterval` 的精度问题和性能浪费
- **拖拽进度条**：支持拖动进度条跳转到任意时间点，内部维护一个当前帧索引

Q：ECharts 和 Recharts 的使用场景分别是什么？

A：

- **ECharts**：在 OMS 项目中使用，优势是功能强大，支持地图热力图、复杂联动、大数据量（`dataZoom` 组件），配置项丰富
- **Recharts**：在全球机会洞察平台中使用，因为项目是 React 技术栈，Recharts 是 React 声明式图表库，用 JSX 写图表更自然，和 React 组件化思想一致
- **选择依据**：跟着主技术栈走，Vue 项目用 ECharts（有 `vue-echarts` 封装），React 项目用 Recharts

1.6 音视频：WebSocket / WASM / requestAnimationFrame / WebGL 渲染

Q：无插件视频流方案的具体实现原理？

A：这是简历中最重要的技术亮点之一，完整链路如下：

摄像头/录像机 → 流媒体服务器 → WebSocket → 前端接收二进制流 → WASM 解码 → 视频帧 → WebGL 渲染到 Canvas

详细步骤：

1. **WebSocket 连接**：与流媒体服务器建立 WebSocket 长连接，接收 H.264/H.265 编码的二进制视频流
2. **WASM 解码**：将 FFmpeg 的解码核心编译为 WebAssembly（.wasm 文件），在浏览器中加载 WASM 模块，把 H.264 码流解码为 YUV 原始帧数据
3. **YUV → RGB 转换**：在 WASM 内部或 WebGL Shader 中做色彩空间转换（YUV420 → RGBA）
4. **requestAnimationFrame 驱动**：用 rAF 做帧调度，每帧从解码缓冲区取出一帧，确保和浏览器刷新率同步，避免掉帧

5. **WebGL 渲染**: 创建 WebGL 纹理 (Texture), 将 RGB 帧数据上传到 GPU, 用 Fragment Shader 渲染到 `<canvas>` 上
6. 为什么用 **WebGL** 而不是 **Canvas 2D**: WebGL 是 GPU 渲染, 比 `ctx.putImageData()` 快一个数量级, 尤其是 1080P 分辨率下

Q: 为什么替代 Windows 插件? 原来的方案有什么问题?

A:

- 原方案是海康的 ActiveX / NPAPI 浏览器插件, 只能在 **Windows + IE/旧版 Chrome** 上运行
- 问题: 不支持 Mac/Linux、不支持新版 Chrome (NPAPI 已被废弃)、安装复杂、安全漏洞风险
- 新方案基于 Web 标准 (WebSocket + WASM + WebGL), **全平台全浏览器兼容**, 用户无需安装任何东西

Q: 延迟大概多少? 怎么处理延迟?

A:

- 端到端延迟约 200-500ms (取决于网络和分辨率)
- **降低延迟的手段**:
 1. 解码缓冲区控制: 只缓存最近 2-3 帧, 丢弃过期帧, 避免累积延迟
 2. 关键帧请求: 当延迟过大时, 向服务端请求 I 帧 (关键帧), 丢弃 P/B 帧等待下一个 GOP
 3. WebSocket 二进制帧传输 (Binary Frame), 比 Base64 编码文本帧体积小 33%

1.7 Node 生态: Node.js / Express / Sequelize / MongoDB / MySQL

Q: Express 后端你是怎么搭建的?

A: 在敏捷排期系统中, 后端架构如下:

```
server/
├─ app.js                // Express 入口, 中间件注册
├─ config/
│   └─ database.js       // 数据库连接配置
├─ models/               // Sequelize 模型定义 (6张表)
│   ├── User.js
│   ├── Sprint.js
│   ├── Task.js
│   ├── Member.js
│   ├── WeeklyReport.js
│   └─ index.js          // 关联关系定义
├─ routes/               // 路由层
├─ controllers/          // 业务逻辑层
└─ middlewares/          // 鉴权、错误处理中间件
```

- 中间件链: `cors` → `body-parser` → `OA OAuth 鉴权` → `路由` → `全局错误处理`
- Sequelize 是 Node.js 最流行的 ORM (对象关系映射) 库, 让你用 JavaScript 对象操作数据库, 而不用手写 SQL。
- Sequelize 定义 6 张表: User、Sprint、Task、Member、WeeklyReport、Team, 通过 `belongsTo/hasMany` 定义关联

- RESTful API 设计: GET /api/sprints、POST /api/sprints/:id/tasks、PUT /api/tasks/:id/status

Q: Sequelize ORM 的关联查询怎么做的?

A:

```
// 查询一个迭代下所有任务及其负责人
const sprint = await Sprint.findByPk(sprintId, {
  include: [
    {
      model: Task,
      include: [
        { model: User, as: "assignee", attributes: ["name", "avatar"] },
      ],
    },
  ],
});
```

- Sequelize 自动将 include 转为 SQL 的 JOIN 语句
- 6 表关联模型: User → Member (多对多, 通过 Member 中间表) → Sprint → Task (一对多) → WeeklyReport

1.8 AI 集成: 阿里云百炼平台 / SSE 流式渲染 / WebSocket AI 对话

Q: 阿里云百炼平台是怎么接入的?

A:

- 阿里云百炼是阿里云的大模型服务平台, 提供通义千问系列模型的 API 调用
- 接入方式: 后端通过 DashScope SDK 调用百炼 API, 传入 system prompt + 用户消息, 获取模型回复
- 前端通过 SSE (Server-Sent Events) 接收流式响应, 逐字渲染

Q: SSE 和 WebSocket 的区别? 为什么 AI 对话用 SSE, 另一个用 WebSocket?

A:

特性	SSE	WebSocket
方向	服务端 → 客户端 (单向)	双向
协议	HTTP	WS/WSS
自动重连	浏览器原生支持	需手动实现
适用场景	服务端持续推送 (如 AI 流式回复)	双向交互 (如聊天、实时通知)

- OMS 运维助理用 SSE: 因为 AI 对话场景是"用户发一条, AI 回一段", 响应方向是单向的, SSE 够用且实现简单

- 全球机会洞察平台用 WebSocket：因为需要双向通信（用户随时中断对话、发送新消息、服务端推送通知），且需要自定义心跳保活

Q：SSE 流式渲染前端怎么实现的？

A：

```
const eventSource = new EventSource("/api/ai/chat?token=xxx");

eventSource.onmessage = (event) => {
  const chunk = JSON.parse(event.data);
  if (chunk.type === "text") {
    // 逐字追加到消息列表
    currentMessage.value += chunk.content;
  } else if (chunk.type === "done") {
    eventSource.close();
  }
};

eventSource.onerror = () => {
  // 错误处理 + 提示用户
  eventSource.close();
};
```

- 关键点：SSE 是 text/event-stream 协议，浏览器原生 EventSource 自动处理重连
- 但 EventSource 只支持 GET 请求，如果需要 POST（发送消息体），需要用 fetch + ReadableStream 手动实现 SSE 客户端
- 实际项目中用 fetch 实现 POST SSE：

```
const response = await fetch("/api/ai/chat", {
  method: "POST",
  body: JSON.stringify({ messages }),
  headers: { "Content-Type": "application/json" },
});
const reader = response.body.getReader();
const decoder = new TextDecoder();
while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  const text = decoder.decode(value);
  // 解析 SSE 格式: "data: {...}\n\n"
  parseSSEChunks(text);
}
```

Q：WebSocket 断线指数退避重连怎么实现的？

A：

```

class ReconnectingWebSocket {
  constructor(url) {
    this.url = url;
    this.retryCount = 0;
    this.maxRetries = 10;
    this.baseDelay = 1000; // 1s
    this.connect();
  }

  connect() {
    this.ws = new WebSocket(this.url);
    this.ws.onopen = () => {
      this.retryCount = 0; // 连接成功，重置重试计数
      this.flushOfflineQueue(); // 发送离线消息
    };
    this.ws.onclose = () => {
      if (this.retryCount < this.maxRetries) {
        const delay =
          this.baseDelay * Math.pow(2, this.retryCount) + Math.random() *
1000;
        // 1s → 2s → 4s → 8s → 16s → ... 最大约 512s
        setTimeout(() => this.connect(), delay);
        this.retryCount++;
      }
    };
  }

  // 离线消息队列
  send(data) {
    if (this.ws.readyState === WebSocket.OPEN) {
      this.ws.send(data);
    } else {
      this.offlineQueue.push(data); // 断线时缓存消息
    }
  }

  flushOfflineQueue() {
    while (this.offlineQueue.length) {
      this.ws.send(this.offlineQueue.shift());
    }
  }
}

```

- 指数退避： $\text{delay} = \text{baseDelay} \times 2^{\text{retryCount}}$ ，避免频繁重连给服务器造成压力
- 随机抖动：加 $\text{Math.random()} \times 1000$ 防止多个客户端同时重连（惊群效应）

- **Token 自动刷新**：在重连前检查 Token 有效期，过期则先调用刷新接口获取新 Token，再用新 Token 建立连接
- **离线消息队列**：断线期间用户的消息暂存内存队列，重连成功后按顺序补发

1.9 其他：GraphQL BFF / DinamicX / OAuth 2.0

Q：GraphQL BFF 数据聚合层是怎么搭建的？

A：

- **BFF (Backend For Frontend)** 是前端和后端之间的中间层，专门为前端服务
- 在 Alibaba.com 项目中，前端需要同时从多个后端服务获取数据：商品服务、供应商服务、运营配置服务、国家供给服务
- 如果直接调 REST API，一个楼层可能需要 3-4 个接口，PC/Mobile/WAP 三端需要的字段还不一样，导致大量冗余请求
- **GraphQL 方案**：
 - 在 BFF 层定义统一的 GraphQL Schema，聚合所有数据源
 - 前端按需查询：PC 端查 `product { name price image }`，Mobile 端只查 `product { name price }`
 - 一次请求获取所需全部数据，无 Over-fetching / Under-fetching

```
query FloorData($floorId: String!) {  
  floor(id: $floorId) {  
    products(first: 20) {  
      name  
      price  
      mainImage  
      supplier {  
        companyName  
        country  
        yearsOnAlibaba  
      }  
    }  
  }  
  operationConfig {  
    bannerUrl  
    themeColor  
  }  
}
```

- BFF 层用 Resolver 分别调用各微服务接口，聚合后返回

Q：DinamicX 动态化方案是什么？具体怎么工作的？

A：

- DinamicX (DX) 是阿里内部的动态化渲染框架，核心思想是"模板和数据分离"
- **工作流程**：

1. 运营在 DX 编辑器中可视化编辑楼层模板（样式、布局、数据绑定规则）
 2. 模板编译后发布到 CDN，生成模板 URL
 3. 服务端下发 模板URL + 业务数据 给前端
 4. 前端 DX SDK 下载模板，将业务数据注入模板，运行时渲染出 UI
- **好处：** 运营可以随意调整楼层样式和内容，前端不需要改代码、不需要发版，实现"运营驱动更新"
 - **三端统一：** DX 模板引擎同时支持 PC（React 渲染）、Mobile App（Native 渲染）、WAP（H5 渲染），一套模板三端通用

Q：OAuth 2.0 单点登录怎么接入的？

A：

- 公司 OA 系统作为 Authorization Server，敏捷排期系统作为 Client
- **接入流程（Authorization Code 模式）：**
 1. 用户访问排期系统，未登录 → 重定向到 OA 登录页
`https://oa.company.com/oauth/authorize?client_id=xxx&redirect_uri=xxx&response_type=code`
 2. 用户在 OA 完成登录（可能是域账号 + 密码）
 3. OA 重定向回排期系统，URL 带 code 参数
 4. 排期系统后端用 code + client_secret 向 OA 换取 access_token
 5. 用 access_token 调用 OA 的用户信息接口，获取 userId、部门、角色
 6. 自动创建/匹配本地用户记录，生成系统 JWT，完成登录
- **组织架构同步：** 登录时从 OA 接口拉取用户所在部门、下属成员，自动同步到本地 User 表，为需求分配提供人员数据

二、项目经历逐条深挖

2.1 Alibaba.com 全球供给首页

Q：三端自适应渲染具体怎么做的？是一套代码还是三套？

A：

- 一套业务代码，通过 ice.js 的多端构建能力，编译出三端产物
- 组件层面：用统一的 props 接口定义楼层数据结构，三端各自实现渲染组件

```
FloorCard/  
├─ index.tsx           // 统一入口，根据 platform 分发  
├─ FloorCard.pc.tsx    // PC 端渲染（宽屏布局，4列商品）  
├─ FloorCard.h5.tsx    // WAP 端渲染（窄屏布局，2列商品）  
└─ FloorCard.weex.tsx  // Mobile App 端（Native 渲染）
```

- 样式层面：PC 用 CSS，WAP 用响应式媒体查询，App 端用 DX 模板
- 核心是"数据协议统一，渲染层分离"

Q：GraphQL Query 和 DX 模板数据的适配层怎么设计的？

A：

- 问题：GraphQL 返回的数据结构是嵌套对象，DX 模板需要的是扁平化的键值对
- 适配层做数据转换：

```
// GraphQL 返回
{
  products: [{ name: "xxx", supplier: { companyName: "ABC" } }];
}

// DX 模板需要
{
  items: [{ title: "xxx", supplierName: "ABC" }];
}
```

- 设计了一套 Schema 映射配置，定义字段映射关系：

```
{
  "items": "$.products",
  "items[].title": "$.products[].name",
  "items[].supplierName": "$.products[].supplier.companyName"
}
```

- 适配层根据当前端类型（PC/Mobile/WAP）选择对应的映射配置，一份 GraphQL Query 驱动三端数据绑定

2.2 全球机会洞察平台

Q：Radix UI + Tailwind CSS 封装组件库，为什么不直接用 shadcn/ui？

A：

- shadcn/ui 确实是基于 Radix + Tailwind 的现成方案，但我们需要：
 - 深度定制设计规范（品牌色、间距、圆角等和公司设计系统对齐）
 - 部分组件 shadcn 没有（如带全球地图联动的 Drawer、带 CSV 预览的 Upload 组件）
 - 组件 API 需要和业务数据模型强绑定
- 所以参考 shadcn/ui 的源码做二次开发，保留底层 Radix 原语（无障碍、键盘导航、焦点管理），上层样式和 API 完全自定义

Q：CSV 批量导入怎么做的？大文件怎么处理？

A：

- 用 `papaparse` 库解析 CSV 文件，支持流式解析大文件
- 流程：
 1. 用户选择 CSV 文件 → `FileReader` 读取
 2. `Papa.parse` 流式解析，逐行触发回调（不一次性加载到内存）
 3. 每 100 行做一次批量校验（字段完整性、格式合法性）
 4. 校验结果实时展示（行号 + 错误信息），用户修正后重新导入

5. 确认无误后批量提交到后端

- 大文件优化：10MB+ CSV 文件用 Web Worker 解析，不阻塞 UI 渲染

Q：Leaflet 全球地图视图怎么做的？

A：

- 使用 `react-leaflet` 封装地图组件
- 营销活动用 Marker 标注在地图上，按国家/地区聚合（MarkerCluster 插件）
- 点击 Marker 弹出 Popover，显示该地区的活动数量、转化率、投入产出比
- 右侧 Recharts 面板展示选中地区的数据图表（柱状图 + 折线图联动）

2.3 OMS 运维服务平台

Q：10W+ 行 Vue 单文件重构为轻量 JS 库，具体怎么做的？

A：

- 原始情况：参数配置模块是一个巨型 Vue 组件（10W+ 行），包含了所有设备类型的参数定义、表单渲染、校验逻辑
- 重构思路：
 1. 将参数定义（字段名、类型、范围、默认值）抽离为纯 JSON 配置 Schema
 2. 将表单渲染逻辑抽离为通用渲染引擎（接收 Schema → 输出 Vue 组件）
 3. 将校验逻辑抽离为独立校验库（基于 JSON Schema 规范）
 4. 最终产出：一个轻量 JS 库（参数 Schema + 渲染引擎 + 校验器），约 5000 行
- 收益：外部团队可以直接引用 JS 库做二次开发，不需要依赖整个 Vue 项目；API 文档用 JSDoc + TypeDoc 自动生成

Q：Vue2 → Vue3 迁移怎么做的？怎么保证业务零感知？

A：

- 迁移策略：渐进式迁移
 1. 构建工具替换：Webpack → Vite（用 `@vitejs/plugin-vue2` 先跑通 Vue 2 项目）
 2. 语法兼容层：引入 `@vue/compat`（兼容模式），它让 Vue 3 运行时支持 Vue 2 的语法
 3. 逐步替换：
 - `$on/$off/$once` 事件总线 → mitt 库
 - `filters` → `computed` 或方法
 - `Vue.prototype` → `app.config.globalProperties`
 - `Vue.set()` → 直接赋值（Vue 3 Proxy 已支持响应式新增属性）
 4. 状态管理：Vuex 3 → Pinia（保持 Options API 写法不变）
 5. UI 组件库：Element UI → Element Plus（组件 API 大部分兼容，少量需手动调整）
- 业务零感知：保留 Options API 开发模式（`data/methods/computed/watch`），不强制迁移到 Composition API，开发者感知不到底层变化
- 最终关闭 `compat` 模式，完成完整迁移

Q：首次加载 7s → 2s 的优化手段有哪些？

A：

1. 路由懒加载：首屏只加载当前页面组件，其他页面按需加载
2. 分包策略：vendor 库独立 chunk，利用浏览器缓存
3. **CDN 外链**：Vue/Element Plus/ECharts 等大库通过 CDN 加载，不走打包（externals 配置）
4. **Gzip/Brotli 压缩**：Vite 配置 vite-plugin-compression，输出 .gz 和 .br 文件，Nginx 直接返回
5. 图片优化：图标用 SVG 雪碧图，大图用 WebP 格式 + 懒加载
6. 预加载：<link rel="prefetch"> 预加载可能访问的下一个页面 chunk
7. **DNS 预解析**：<link rel="dns-prefetch" href="//api.xxx.com">

Q：异步下载队列（轮询状态机 + 进度追踪）怎么设计的？

A：

- 问题：用户可能同时下载 4-8 路视频，需要控制并发、追踪进度、处理失败重试
- 状态机设计：

```
graph LR
  IDLE --> QUEUED
  QUEUED --> DOWNLOADING
  DOWNLOADING --> COMPLETED
  DOWNLOADING --> FAILED
  FAILED --> RETRYING
  RETRYING --> QUEUED
```

- 核心实现：
 - 最大并发数 3（可配置），超出的任务进入等待队列
 - 每个任务有独立的进度条（已下载字节 / 总字节）
 - 轮询服务端下载状态（因为视频下载是服务端异步任务，前端需要轮询）
 - 失败自动重试 3 次，重试间隔指数退避
 - 全局进度 = 所有任务进度的加权平均

2.4 敏捷迭代排期系统

Q：资源负载可视化面板怎么实现的？

A：

- 后端按成员维度聚合：

```
SELECT member_id, SUM(estimated_hours) as total_hours
FROM tasks
WHERE sprint_id = ? AND status != 'done'
GROUP BY member_id
```

- 计算负载率：loadRate = total_hours / (workdays × 8h)
- 前端用颜色梯度展示：
 - 绿色 (< 80%)：负载健康
 - 黄色 (80%-100%)：接近满载
 - 红色 (> 100%)：超载预警，闪烁提示
- 支持拖拽任务卡片在不同成员之间移动，实时更新负载率

Q：Vite 按需导入打包体积 -60% 怎么做到的？

A:

- Element Plus 全量引入约 2.5MB (JS + CSS)，按需引入后只用到的组件 (约 20 个) 被打包
 - 配合 `unplugin-auto-import` (自动导入 Vue/Router/Pinia API) 和 `unplugin-vue-components` (自动注册组件)
 - 路由懒加载 + 分包: 首屏只加载 `vue-router` 当前路由对应的 chunk，其他页面异步加载
 - 最终效果: 首屏核心包约 200KB (gzip 后)，总体积从全量 2.5MB 降至约 1MB
-

三、工作经历 & 管理相关

Q: 你是怎么组建前端团队的?

A:

- 在大华组建 5 人团队: 从面试选人 → 制定规范 → 分配方向。5 人分别负责: 2 人做核心业务模块、1 人做公共组件库、1 人做数据可视化、1 人做测试和文档
- 在海康组建 3 人团队: 1 人做 GIS 地图模块、1 人做视频监控模块、我做核心业务 + 技术架构

Q: 需求优先级怎么排的?

A:

- 用 Eisenhower 矩阵分类: 紧急且重要 (线上 Bug) → 重要不紧急 (技术重构) → 紧急不重要 (临时运营需求) → 不紧急不重要
- 和 PM 对齐: 每周需求评审会, 用故事点 (Story Point) 评估工作量, 按迭代容量排优先级
- 在敏捷排期系统中实现了这套流程, PM 可以直接在系统里拖拽排期

Q: 你觉得你最大的技术亮点是什么?

A:

- 无插件视频流方案: WebSocket + WASM + WebGL 的完整链路, 是真正从底层解决问题的方案, 解决了浏览器插件淘汰后的技术断层
 - **GraphQL + DX 的多端动态化**: 一套 Schema 驱动三端, 在 Alibaba.com 这种全球化大流量场景下验证了方案的可行性
 - **性能优化的系统性思维**: 不是单纯调一个参数, 而是从构建、网络、渲染三个维度系统性优化, 7s → 2s 是综合施策的结果
-

本文档覆盖了简历中所有可深挖的技术点, 建议逐一准备, 面试时根据问题灵活组合回答。